

MATH 231 – Linear Algebra I

Project 1 ■ Neighbours, Clusters, and Valleys

Kiely Hall 326 ■ Instructor: Kevin Bedoya

Due: Sunday, November 1, 2026, 11:59 PM.

Three of the most-used algorithms in data science are built out of nothing but the vector operations of this course: distances, means, and gradients. This project offers all three, as three separate choices, and **you choose one**. In the choice you make, you write the algorithm yourself in plain NumPy, watch it work, and write a short report on what you saw. You can teach a computer to **name a song's genre** from its nearest neighbours, decide where to **park four food trucks** in a city, or roll **marbles** down a landscape with eight valleys and watch them move in 3-D as your code runs.

Choice	Algorithm	TRACK	Lectures	You write
1	k -nearest neighbours	"knn"	computational unit §5–§8; vector unit §14	4 functions
2	k -means (Lloyd's algorithm)	"kmeans"	computational unit §11–§14	4 functions
3	gradient descent	"descent"	computational unit §24–§27; vector calculus unit §6–§8	2 functions

Each choice is about 45 minutes of code and 30 minutes of writing, for someone who has done the NumPy part of Homework 1. Every algorithm is stated in full in a box in its choice, so this handout is all you need, but the lecture notes cited beside each box say more about *why* each one works.

Making your choice

- Pick whichever choice interests you. The three are graded the same way and are worth the same.
- Read these opening pages, then go straight to your choice. Each choice is self-contained: its story, its algorithm, its steps, and, at its end, the questions for its report.
- Name your choice in `project1.py` by setting `TRACK` to `"knn"`, `"kmeans"` or `"descent"`. `runproject1.py` then runs only your choice, and only that choice is graded.

What you receive

A folder `projects/project1/starter/` in the course repository. Copy the whole folder into your own repository. It holds five files:

File	You...	What it is
<code>project1.py</code>	edit	<code>TRACK</code> , and the functions for all three choices, each with a docstring saying exactly what it takes and returns. You set <code>TRACK</code> and write your choice's functions. The only file you change.
<code>runproject1.py</code>	run	Runs your choice's steps in order, prints labelled results, saves figures in <code>figures/</code> . This is what the grader runs.
<code>play.py</code>	run	The live animations for Choices 2 and 3. For you, not graded.
<code>data.py</code>	—	The data sets, generated from fixed seeds so that everyone has the same numbers.
<code>viz.py</code>	—	The plotting and animation code.

What to submit

Push the folder to your MATH 231 repository as `projects/project1/`, with three things in it besides the provided files:

- your completed `project1.py`, with `TRACK` set, and your name and your generative-AI disclosure filled in at the top (the syllabus, §9.2, asks for it: which tool, and what for, or “none”);
- the `figures/` folder that `runproject1.py` creates;
- `report.pdf`, your written report on your choice (see *The report*, below). PDF is the only accepted format, as for homework.

Python and NumPy only — no scikit-learn, no SciPy. The point is to write the algorithm, not to call it.

How it is graded

Your choice is graded out of 100.

Code, 55 points. The grader runs `runproject1.py` exactly as you submitted it and reads its output, step by step. `runproject1.py` labels every result for you (Output of step 2.4 :), so there is nothing to label in the code. The points per function are:

Choice	Functions and points
1	distance 10, knn_predict 20, accuracy 10, standardize 15
2	assign 15, update 15, wcss 10, kmeans 15
3	grad_f 20, gradient_descent 35

Report, 45 points. Questions R1 and R2 of your choice at 18 points each, and its **Limits** paragraph at 9.

The report

One to three pages, including figures, as `report.pdf`, answering the report questions at the end of your choice. It is graded on whether your answers are correct, specific, and supported by your own output: quote the numbers `runproject1.py` printed, and include the figures you refer to. Label the answers **R1**, **R2** and **Limits**, in that order.

Every choice ends with a **Limits** question, because the syllabus asks every project report to say what the limits of its method are.

Getting started

You need NumPy and matplotlib (Setup Guide 2 covers installing packages):

```
python -m pip install numpy matplotlib
```

Open `project1.py` and set `TRACK` near the top, for example

```
TRACK = "kmeans"
```

Then, inside the `starter` folder, run `runproject1.py` straight away, before writing anything:

```
python runproject1.py
```

It names your choice, and every step in it prints `not implemented yet`. That is the starting line. (If `TRACK` is not set, `runproject1.py` runs all three choices and reminds you to set it.) Work through your steps in order, and run `runproject1.py` after each function you write: the step you just finished will start printing results, and each **Checkpoint** below tells you what the correct result is. A function that crashes is reported as **ERROR** with the message, and the rest still runs.

Watch out. Three rules for `project1.py`. Do not rename a function or change its arguments — `runproject1.py` calls them by name. Apart from the `TRACK` line, put nothing at the left margin that runs or prints: definitions only. And leave `runproject1.py`, `data.py` and `viz.py` unchanged; the grader uses the original copies.

Choice 1. Name that genre: k -nearest neighbours

Track: TRACK = "knn". **You write:** distance, knn_predict, accuracy, standardize.

A music app describes every song by two numbers, its **duration** in seconds and its **energy** on a scale from 0 to 1, so each song is a point in $\mathbb{R}^2$. It already knows the genre of 150 songs (lo-fi, pop, or metal), and it has to guess the genre of 90 new ones. You will measure how often it guesses right.

The algorithm: k -NN classification (computational unit §6)

Given labelled training points $\mathbf{x}_1, \dots, \mathbf{x}_N$ with labels $y_1, \dots, y_N$, a query point $\mathbf{q}$, a metric d , and a whole number k :

1. **Measure.** Compute $d(\mathbf{q}, \mathbf{x}_i)$ for every i .
2. **Select.** Find the k training points with the smallest distances: the k nearest neighbours.
3. **Vote.** Predict the label that occurs most often among them. If labels tie, go through the neighbours from nearest to farthest and take the first label that has the top vote count.

The distances are the p -norm distances of vector unit §14: $d_1(\mathbf{x}, \mathbf{y}) = \sum_i |x_i - y_i|$, $d_2(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_i (x_i - y_i)^2}$, $d_\infty(\mathbf{x}, \mathbf{y}) = \max_i |x_i - y_i|$.

Step 1.1 (distance). Write `distance(x, y, p=2)` for $p = 1, 2$ and `np.inf`, using `np.abs`, `np.sum`, `np.max` and `np.sqrt` — not `np.linalg.norm`.

Checkpoint. `runproject1.py` prints the three distances from $[1, 1]$ to $[4, 5]$: 7, 5 and 4.

Step 1.2 (knn_predict). Write `knn_predict(X_train, y_train, q, k, p=2)`, following the box. `X_train` holds one training point per row, so `X_train[i]` is the i -th point and `y_train[i]` its label.

Hint. `np.argsort(dists)` returns the indices that would sort `dists` from smallest to largest, so `np.argsort(dists)[:k]` are the indices of the k nearest neighbours. A Python dictionary is a convenient way to count votes.

Checkpoint. `runproject1.py` runs the worked example of computational unit §6.5, eight points and the query $[5, 5]$. It should print A for $k = 1$ and B for $k = 3$ and $k = 5$. Your report explains why they disagree (R1).

Step 1.3 (accuracy). Write `accuracy(X_train, y_train, X_test, y_test, k, p=2)`: predict every test song with `knn_predict` and return the fraction predicted correctly. `runproject1.py` reports it for $k = 1, 3, 5, 9, 15$ on the raw song data.

Checkpoint. Every accuracy lands between 0.41 and 0.49. Guessing at random would score 0.33. Something is badly wrong, and it is not your code: see Step 1.4.

Step 1.4 (standardize). Look at the numbers. A duration is a few hundred seconds, and two songs' durations routinely differ by 50. An energy is between 0 and 1, so two songs' energies differ by at most 1. In d_2 the duration gap is squared along with the energy gap, and it swamps it: the classifier is effectively sorting songs by *length*, which says almost nothing about genre. This is computational unit §8.2, "Feature scaling, and why it is not optional".

The fix is to measure each feature in units of its own spread. Write `standardize(X_train, X_test)`: compute the mean and standard deviation of each feature from the *training* songs only, and return both sets rescaled as $(\text{value} - \text{mean})/\text{sd}$.

Hint. `X_train.mean(axis=0)` averages down each column, giving one mean per feature. `X_train.std(axis=0)` does the same for the standard deviation. Subtracting the one and dividing by the other, as whole arrays, rescales every row at once.

Checkpoint. `runproject1.py` prints the scaled training means (0) and standard deviations (1), then the accuracies again. They should all be close to 0.9.

Step 1.5 (Look at the pictures — nothing to write). `runproject1.py` now compares the three metrics at $k = 9$ and saves three *decision-region* pictures: every point of the plane coloured by the genre your classifier would give a song there. Open them in `figures/`: `knn_raw_k15.png`, `knn_scaled_k1.png` and `knn_scaled_k15.png`. You will use `knn_raw_k15.png` in R2.

The report for Choice 1

R1. The worked example (Step 1.2). For the query `[5, 5]`, give the Euclidean distance to each of the eight training points, and list the 1, 3 and 5 nearest neighbours with their labels. Use them to explain why $k = 1$ predicts A while $k = 3$ and $k = 5$ predict B.

R2. Scaling (Steps 1.3–1.4). Give the raw and scaled accuracies as a table. Explain why scaling helped so much, using the actual spreads of the two features (`runproject1.py` prints the raw training standard deviations in Step 1.4). Then look at `knn_raw_k15.png`: why are its boundaries almost vertical lines?

Limits. Write a short paragraph on the limits of k -NN: one thing it cannot do, or does badly, that you saw in this project, and one more that you did not see here but would expect on real data. Computational unit §8, “Choosing k , and what goes wrong”, is a good place to look.

Choice 2. Park the food trucks: k -means

Track: TRACK = "kmeans". **You write:** assign, update, wcss, kmeans.

A food-truck company has 320 regular customers, and it knows where each one lives. It owns four trucks. Where should it park them so that customers do not have far to walk? One precise version of the question is to choose four parking spots $\mu_1, \dots, \mu_4$ that make

$$J = \sum_{\text{customers } \mathbf{x}} \|\mathbf{x} - (\text{nearest parking spot})\|_2^2$$

as small as possible. That is the k -means problem with $k = 4$, and J is its **within-cluster sum of squares** (computational unit §12.1).

The algorithm: Lloyd's k -means (computational unit §12.3)

Given points $\mathbf{x}_1, \dots, \mathbf{x}_N$ and initial centres $\mu_1, \dots, \mu_k$, called a **start** (a **random start** takes k different data points, chosen at random, as the initial centres):

1. **Assign.** Give each point the label of its nearest centre, comparing squared distances $\|\mathbf{x}_i - \mu_j\|_2^2$.
2. **Update.** Move each centre to the mean of the points labelled with it. (If none are, leave it where it is.)
3. Repeat from 1 until an assignment step changes no labels.

Neither step can increase J , and that is why the algorithm stops (computational unit §12.4). It stops at a *local* minimum, which need not be the best one (§14.1).

Step 2.1 (assign). Write `assign(X, centres)`, returning an integer array of labels: `labels[i]` is the index of the centre nearest to `X[i]`.

Checkpoint. `runproject1.py` runs the six-point example of computational unit §12.5, with its deliberately bad start $\mu_1 = [1, 1]$, $\mu_2 = [2, 1]$. The first assignment is `[0 1 0 1 1 1]`.

Step 2.2 (update). Write `update(X, labels, centres)`, returning a *new* array of centres, each the mean of its cluster.

Hint. `X[labels == j]` picks out the rows of `X` labelled `j`.

Checkpoint. After the first update the centres are `[1, 1.5]` and `[6.75, 6.5]`.

Step 2.3 (wcss). Write `wcss(X, labels, centres)`, returning J .

Checkpoint. $J = 284$ at the start of the example, and $J = 72.25$ after the first update.

Step 2.4 (kmeans). Write `kmeans(X, init_centres, max_iter=100)`: alternate `update` and `assign` until the labels stop changing. Return the centres, the labels, and `history`, the list of J values recorded after each update.

Checkpoint. The six-point example ends with labels `[0 0 0 1 1 1]`, centres $[\frac{4}{3}, \frac{4}{3}]$ and $[\frac{25}{3}, \frac{25}{3}]$, and history `[72.25, 2.6667]`. On the city, `runproject1.py` checks that J never increased and draws the four clusters; the final J is about 442.

Step 2.5 (Executing k -means). With your four functions written, run `runproject1.py` again. Step 2.5 runs your `kmeans` on the city three more ways:

- from a bad start, with three trucks downtown and one at the university; the result is saved as `figures/kmeans_city_bad_start.png`;
- from ten random starts, printing the final J of each and the best of the ten;
- for every k from 1 to 8, keeping the best of ten random starts each time, to draw the **elbow plot** of computational unit §14.2: the smallest J found for each k , plotted against k . The curve falls steeply while k is too small and then flattens out; the **elbow** is the value of k where it bends from steep to flat. The plot is saved as `figures/kmeans_elbow.png`.

Then open the two figures and read the printed output. Note where the bad start's four trucks ended up compared with the four neighbourhoods, how many of the ten random starts finished with the same J as the bad start, and the value of k at which the elbow plot bends. Your report (R1 and R2) asks about all three.

Watch it live. `python play.py` animates the bad start and then a random start, one assignment and one update at a time.

The report for Choice 2

- R1. Bad starts and restarts (Step 2.5).** Include `kmeans_city_bad_start.png` and describe what went wrong there. How many of the ten random starts ended the same way? What does keeping the best of ten starts buy you, and what does it cost?
- R2. The elbow (Step 2.5).** Include `kmeans_elbow.png`. How many trucks does the data suggest, and why not simply take the k with the smallest J ?

Limits. Write a short paragraph on the limits of k -means: one thing it cannot do, or does badly, that you saw in this project, and one more that you did not see here but would expect on real data. Computational unit §14, “What goes wrong”, is a good place to look.

Choice 3. Marbles in the valleys: gradient descent

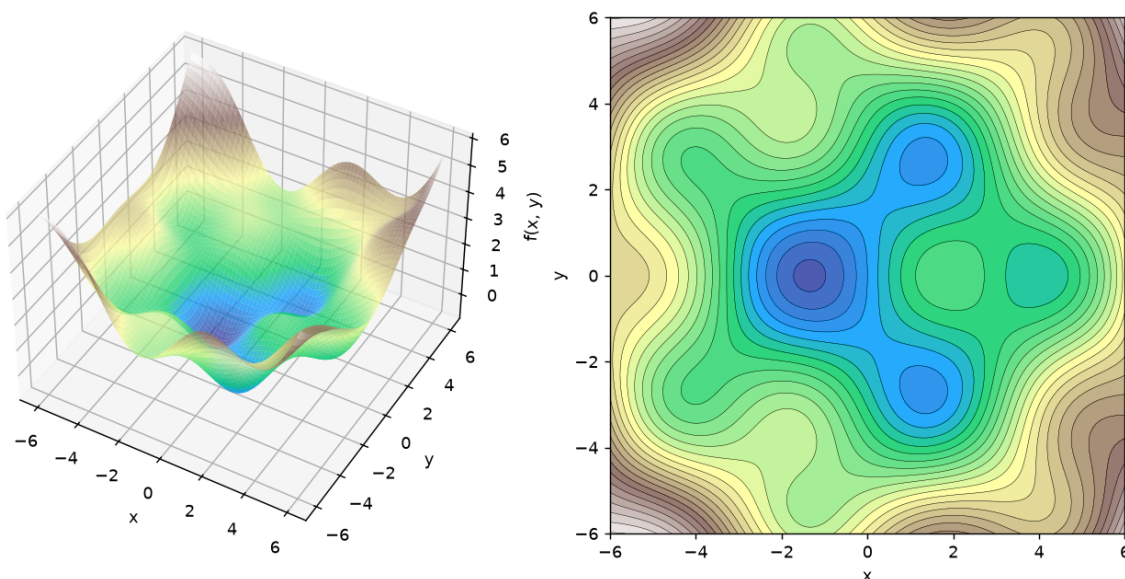
Track: TRACK = "descent". **You write:** grad_f, gradient_descent.

Here is a landscape: the graph of

$$f(x, y) = \sin x \cos y + 0.08(x^2 + y^2),$$

a wavy egg-crate tipped into a gentle bowl. It has eight valleys with floors in the square $-6 \leq x, y \leq 6$, all of different depths except for a few matched pairs. Drop a marble anywhere and it rolls downhill until it settles on some valley floor, and which floor depends on where you dropped it. Gradient descent is that marble.

The landscape



The algorithm: gradient descent (computational unit §27.4)

Given the gradient ∇f , a start $\mathbf{x}_0$, a learning rate $\eta > 0$, a tolerance τ and a cap on the number of steps:

1. Compute $\mathbf{g} = \nabla f(\mathbf{x})$.
2. If $\|\mathbf{g}\| < \tau$, stop: the ground is flat, report success.
3. Otherwise step downhill, $\mathbf{x} \leftarrow \mathbf{x} - \eta \mathbf{g}$, and go back to 1 — unless the cap has been reached, in which case report failure.

It works because ∇f points in the direction of steepest ascent (vector calculus unit §8.3), so $-\nabla f$ points steepest downhill. The learning rate decides how far to go: too small and the marble crawls, too large and it overshoots (computational unit §25.4).

Step 3.1 (grad_f). Compute $\partial f/\partial x$ and $\partial f/\partial y$ by hand (vector calculus unit §6.3: hold the other variable constant). Write your derivation in the report (R1), then code `grad_f(p)` to return $\nabla f = [\partial f/\partial x, \partial f/\partial y]$ as a NumPy array.

Checkpoint. $\nabla f(0,0) = [1, 0]$. `runproject1.py` also compares your gradient with a numerical one (built from the limit definition) at 50 random points; the largest disagreement should be around 10^{-9} . If it is anywhere near 1, a partial derivative is wrong.

Step 3.2 (`gradient_descent`). Write

```
gradient_descent(grad, x0, lr, tol=1e-6, max_iter=1000)
```

following the box, taking at most `max_iter` steps. Return `path`, the list of every point visited starting with `x0`, and `converged`, which is `True` if the box's test $\|g\| < \tau$ passed.

Watch out. Build each new point as a new array, as `x = x - lr * g` does, or append a *copy* of it to the path. Updating one array in place with `x -= lr * g` would leave every entry of your path pointing at the same, final, point.

Checkpoint. `runproject1.py` first runs the example of computational unit §27.2, $f(x,y) = x^2 + y^2$ from $[1, 2]$ with $\eta = 0.1$. The path begins $[1, 2]$, $[0.8, 1.6]$, $[0.64, 1.28]$, $[0.512, 1.024]$, and converges after 69 steps. With $\eta = 1$ it bounces between $[1, 2]$ and $[-1, -2]$ forever, and after the cap of 20 steps it is back at $[1, 2]$ with `converged False` — the row $\eta = 1$ of the table in §25.4.

Step 3.3 (Five marbles — nothing to write). `runproject1.py` drops five marbles with $\eta = 0.3$ and reports where each one settled, how deep its valley is, and how many steps it took.

Checkpoint. The first marble, dropped at $(0.5, 0.5)$, settles at about $(-1.353, 0)$ with $f \approx -0.830$ after 39 steps.

Watch it live. `python play.py marbles` shows all five rolling at once, on the 3-D surface and on the contour map.

Step 3.4 (Learning rates — nothing to write). From the single start $(3, 1.5)$ `runproject1.py` tries $\eta = 0.01, 0.1, 0.5, 1, 1.5$ and 2 , and reports which runs converged and how fast. Then it drops one marble at $(-4, 4.6)$ twice, with $\eta = 0.1$ and with $\eta = 1.5$.

Watch it live. `python play.py rates` races those last two marbles.

The report for Choice 3

R1. The gradient (Step 3.1). Show your derivation of $\partial f/\partial x$ and $\partial f/\partial y$, naming the rule you use at each step. Then evaluate your gradient at $(0, 0)$ by hand, and quote the largest disagreement with the numerical gradient that `runproject1.py` found.

R2. Marbles and learning rates (Steps 3.3–3.4). Which marble found the deepest valley? Two marbles took about ten times as many steps as the others: look at the contour map near where they stopped and explain what is different about those valleys. Make a table of the learning-rate runs and explain each kind of behaviour you see. Finally, the marble at $(-4, 4.6)$ reached the deepest valley only with the larger learning rate. Is that a reason to use large learning rates?

Limits. Write a short paragraph on the limits of gradient descent: one thing it cannot do, or does badly, that you saw in this project, and one more that you did not see here but would expect on real problems. Computational unit §26.2, “What it does not promise”, is a good place to look.

Optional and ungraded

- **Try another choice.** `python runproject1.py kmeans` runs Choice 2 whatever **TRACK** says, and likewise `knn`, `descent`, or `all`. Only the choice named by **TRACK** is graded.
- **Paint by numbers** (Choice 2). Load a photo with `plt.imread`, treat every pixel as a point [red, green, blue] in $\mathbb{R}^3$, run your `kmeans` with $k = 8$ on a random sample of a few thousand pixels, and repaint every pixel with the colour of its nearest centre. Your `assign` works unchanged in $\mathbb{R}^3$. Why?
- **Make a GIF** (Choice 3). `viz.watch_descent` takes a `save_frames` argument that saves every frame as a picture; any GIF maker will stitch them together for your report.

Questions about the project belong in the class Discord, where everyone can see the answer. Questions about your own code are better brought to office hours, Thursdays 3:00–4:00 PM in Kiely Hall 508. Please email ahead.